Lecture 4: The Complexity Class **P**

Dr. George Mertzios

george.mertzios@durham.ac.uk

Room 2066, MCS Building
Tel: 42429

The Big–Oh Notation - Reminder

Definition

A function $f(n)$ is said to be $O(g(n))$ if and only if there are positive constants, c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$.

Intuition: f grows not faster than g.

Exercise

1. $3n + 7$ is $O(n)$?
2. $n^2 + n$ is $O(n^2)$?
3. n^2 is $O(n^2 + n)$?
4. n is $O(0.1n^2)$?
5. $0.1n^2$ is $O(n)$?
6. $\log n$ is $O(n)$?
7. n is $O(\log n)$?
8. n^k is $O(2^n)$ (for fixed k)?
9. 2^n is $O(n^k)$ (for fixed k)?

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$?
3. n^2 is $O(n^2 + n)$?
4. n is $O(0.1n^2)$?
5. $0.1n^2$ is $O(n)$?
6. $\log n$ is $O(n)$?
7. n is $O(\log n)$?
8. n^k is $O(2^n)$ (for fixed k)?
9. 2^n is $O(n^k)$ (for fixed k)?

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$?
3. n^2 is $O(n^2 + n)$?
4. n is $O(0.1n^2)$?
5. $0.1n^2$ is $O(n)$?
6. $\log n$ is $O(n)$?
7. n is $O(\log n)$?
8. n^k is $O(2^n)$ (for fixed k)?
9. 2^n is $O(n^k)$ (for fixed k)?

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$? YES
3. n^2 is $O(n^2 + n)$?
4. n is $O(0.1n^2)$?
5. $0.1n^2$ is $O(n)$?
6. $\log n$ is $O(n)$?
7. n is $O(\log n)$?
8. n^k is $O(2^n)$ (for fixed k)?
9. 2^n is $O(n^k)$ (for fixed k)?

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$? YES
3. n^2 is $O(n^2 + n)$? YES
4. n is $O(0.1n^2)$?
5. $0.1n^2$ is $O(n)$?
6. $\log n$ is $O(n)$?
7. n is $O(\log n)$?
8. n^k is $O(2^n)$ (for fixed k)?
9. 2^n is $O(n^k)$ (for fixed k)?

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$? YES
3. n^2 is $O(n^2 + n)$? YES
4. n is $O(0.1n^2)$? YES
5. $0.1n^2$ is $O(n)$?
6. $\log n$ is $O(n)$?
7. n is $O(\log n)$?
8. n^k is $O(2^n)$ (for fixed k)?
9. 2^n is $O(n^k)$ (for fixed k)?

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$? YES
3. n^2 is $O(n^2 + n)$? YES
4. n is $O(0.1n^2)$? YES
5. $0.1n^2$ is $O(n)$? NO
6. $\log n$ is $O(n)$?
7. n is $O(\log n)$?
8. n^k is $O(2^n)$ (for fixed k)?
9. 2^n is $O(n^k)$ (for fixed k)?

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$? YES
3. n^2 is $O(n^2 + n)$? YES
4. n is $O(0.1n^2)$? YES
5. $0.1n^2$ is $O(n)$? NO
6. $\log n$ is $O(n)$? YES
7. n is $O(\log n)$?
8. n^k is $O(2^n)$ (for fixed k)?
9. 2^n is $O(n^k)$ (for fixed k)?

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$? YES
3. n^2 is $O(n^2 + n)$? YES
4. n is $O(0.1n^2)$? YES
5. $0.1n^2$ is $O(n)$? NO
6. $\log n$ is $O(n)$? YES
7. n is $O(\log n)$? NO
8. n^k is $O(2^n)$ (for fixed k)?
9. 2^n is $O(n^k)$ (for fixed k)?

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$? YES
3. n^2 is $O(n^2 + n)$? YES
4. n is $O(0.1n^2)$? YES
5. $0.1n^2$ is $O(n)$? NO
6. $\log n$ is $O(n)$? YES
7. n is $O(\log n)$? NO
8. n^k is $O(2^n)$ (for fixed k)? YES
9. 2^n is $O(n^k)$ (for fixed k)?

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$? YES
3. n^2 is $O(n^2 + n)$? YES
4. n is $O(0.1n^2)$? YES
5. $0.1n^2$ is $O(n)$? NO
6. $\log n$ is $O(n)$? YES
7. n is $O(\log n)$? NO
8. n^k is $O(2^n)$ (for fixed k)? YES
9. 2^n is $O(n^k)$ (for fixed k)? NO

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$? YES
3. n^2 is $O(n^2 + n)$? YES
4. n is $O(0.1n^2)$? YES
5. $0.1n^2$ is $O(n)$? NO
6. $\log n$ is $O(n)$? YES
7. n is $O(\log n)$? NO
8. n^k is $O(2^n)$ (for fixed k)? YES
9. 2^n is $O(n^k)$ (for fixed k)? NO

Question 1: compare growth rate of 2^n and 3^n;

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$? YES
3. n^2 is $O(n^2 + n)$? YES
4. n is $O(0.1n^2)$? YES
5. $0.1n^2$ is $O(n)$? NO
6. $\log n$ is $O(n)$? YES
7. n is $O(\log n)$? NO
8. n^k is $O(2^n)$ (for fixed k)? YES
9. 2^n is $O(n^k)$ (for fixed k)? NO

Question 1: compare growth rate of 2^n and 3^n; $2^n = o(3^n)$

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$? YES
3. n^2 is $O(n^2 + n)$? YES
4. n is $O(0.1n^2)$? YES
5. $0.1n^2$ is $O(n)$? NO
6. $\log n$ is $O(n)$? YES
7. n is $O(\log n)$? NO
8. n^k is $O(2^n)$ (for fixed k)? YES
9. 2^n is $O(n^k)$ (for fixed k)? NO

Question 1: compare growth rate of 2^n and 3^n; $2^n = o(3^n)$
Question 2: what about $\log_2 n$ and $\log_{999} n$?

Exercise

1. $3n + 7$ is $O(n)$? YES
2. $n^2 + n$ is $O(n^2)$? YES
3. n^2 is $O(n^2 + n)$? YES
4. n is $O(0.1n^2)$? YES
5. $0.1n^2$ is $O(n)$? NO
6. $\log n$ is $O(n)$? YES
7. n is $O(\log n)$? NO
8. n^k is $O(2^n)$ (for fixed k)? YES
9. 2^n is $O(n^k)$ (for fixed k)? NO

Question 1: compare growth rate of 2^n and 3^n; $2^n = o(3^n)$
Question 2: what about $\log_2 n$ and $\log_{999} n$?
$\log_2 n = \log_2 999 \cdot \log_{999} n \quad \Rightarrow \quad \log_2 n = \Theta(\log_{999} n)$

Time Complexity

Definition

For any function f, we say that the time complexity of a decidable language $\mathcal{L}$ is $O(f)$, or $\mathcal{L}$ is decidable in $O(f)$ time, if there exists a TM T which decides $\mathcal{L}$, and constants n_0 and c such that for all inputs x with $|x| > n_0$,

$$Time_T(x) \leq c \cdot f(|x|)$$

Complexity Classes

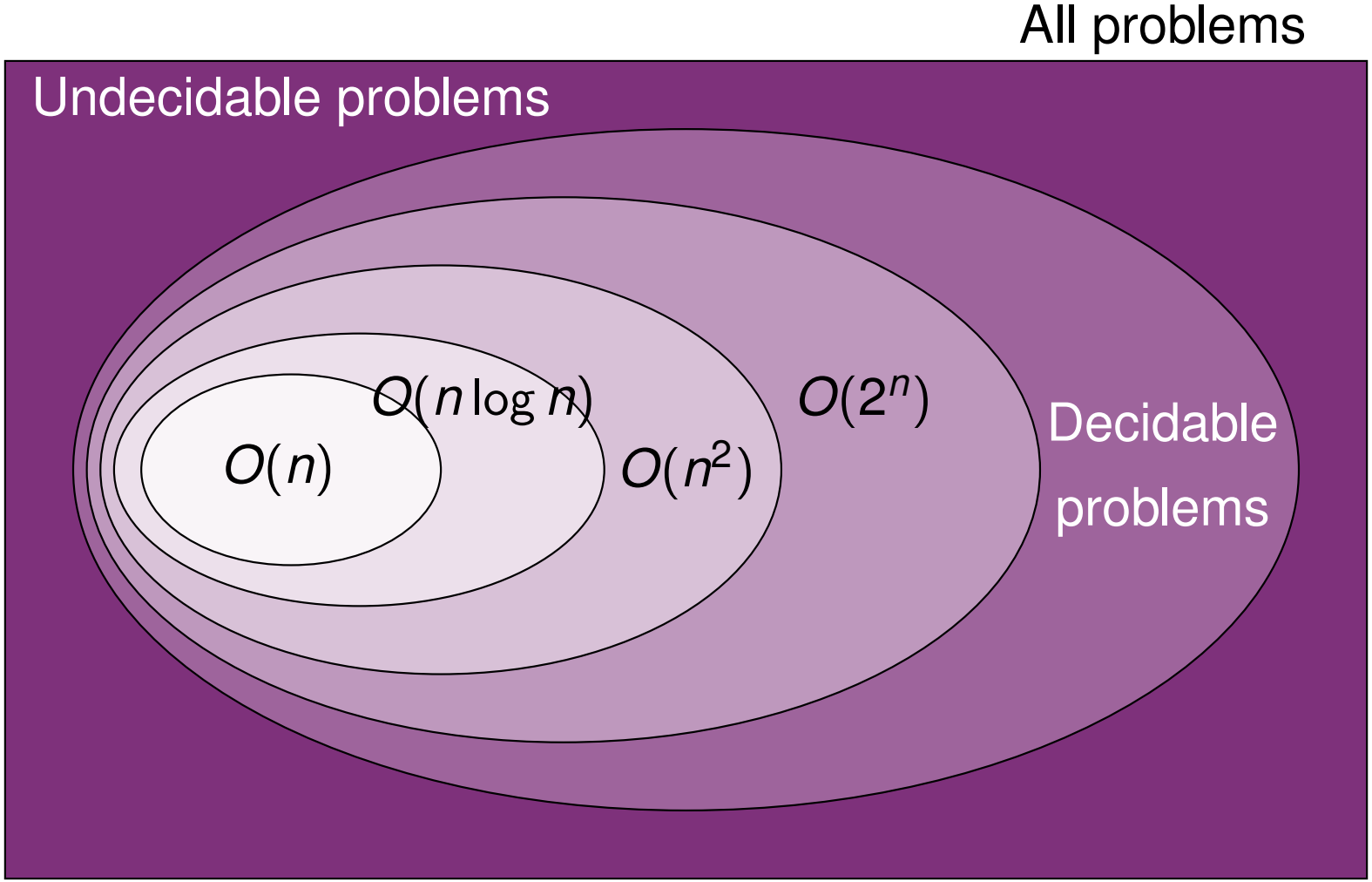
Now we are in a position to divide up the decidable languages into classes, according to their time complexity.

Definition

The time complexity class TIME[f] is defined to be the class of all problems (i.e. languages), for which there exists an algorithm with time complexity in $O(f)$.

- it is sometimes called DTIME[f] – for Deterministic Time

Examples


All problems
Undecidable problems
$O(n \log n)$
$O(2^n)$
Decidable problems
$O(n)$
$O(n^2)$

The Complexity Class **P**

Definition

$$\mathbf{P} = \bigcup_{k \geq 0} TIME[n^k]$$

Note: The complexity class **P** contains all classes $TIME[t(n)]$, where $t(n)$ is a polynomial of finite degree.

The Complexity Class **P**

Time Complexity	Size n					
	10	20	30	40	50	60
n	.00001 seconds	.00002 seconds	.00003 seconds	.00004 seconds	.00005 seconds	.00006 seconds
n^2	.0001 seconds	.0004 seconds	.0009 seconds	.0016 seconds	.0025 seconds	.0036 seconds
n^3	.001 seconds	.008 seconds	.027 seconds	.064 seconds	.125 seconds	.216 seconds
n^5	.1 seconds	3.2 seconds	24.3 seconds	1.7 minutes	5.2 minutes	13.0 minutes
2^n	.001 seconds	1.0 seconds	17.9 minutes	12.7 days	35.7 years	366 centuries
3^n	.059 seconds	58 minutes	6.5 years	3855 centuries	2×10^8 centuries	1.3×10^{13} centuries

$\mathbf{P}$ = doable?

The class **P** is a reasonable mathematical model of the class of problems which are tractable or solvable in practice.

However, the correspondence is not exact:

- When the degree of the polynomial is high then the time grows so fast that in practice the problem is not solvable
- The constants may also be very large, for example $2^{2^{2^{2^{2^{10}}}}}$

(These problems usually seem to arise only in artificially created problems; for naturally defined problems the polynomials and constants are not too large)

Different Models of Computation

Lemma

We can simulate t steps of k-tape TM with an equivalent one-tape TM in $O[t^2]$ steps

Lemma

We can simulate t steps of a two-way infinite k-tape machine with an equivalent k-tape TM in $O[t]$ steps

Hence the class **P** is the same for all of these models of computation (and many others).

Different Encodings

Lemma

For any number n, the length of the encoding of n in base b_1 and the length of the encoding of n in base b_2 are related by a constant factor (provided $b_1, b_2 \geq 2$)

- What if we encode n in base $b_0 = 1$?

Different Encodings

Lemma

For any number n, the length of the encoding of n in base b_1 and the length of the encoding of n in base b_2 are related by a constant factor (provided $b_1, b_2 \geq 2$)

- What if we encode n in base $b_0 = 1$?

Lemma

For any graph G, the length of the encoding of G as an adjacency matrix, and the length of the encoding of G as a list of edges, are both related by a polynomial factor to the number of vertices.

- Which is this polynomial factor?

Hence the class **P** is the same for these encodings (and many others).

Proving a problem is in **P**

- The class **P** is said to be robust — it does not depend on the exact details of the computational model or encoding.

$\Rightarrow$ we do not need to specify all the details of the machine model or even the encoding (e.g. pseudo-code is sufficient).

- The most direct way to show that a problem is in **P** is to give a polynomial time algorithm which solves it.
- Even a naive polynomial-time algorithm often provides a good insight into how the problem can be solved efficiently.
- To find such an algorithm we generally need to identify an approach to the problem that is considerably better than brute-force search.

CNF

Some of the most important computational problems concern logical formulas

- A logical formula f is said to be in conjunctive normal form (CNF) if

$$f = C_1 \wedge \ldots \wedge C_m$$

  where each C_i is a clause, which is a disjunction ($\vee$, "OR") of literals:

$$C_i = (l_{i1} \vee \ldots \vee l_{ik})$$

  and a literal is either a variable or its negation, e.g. x_i or $\neg x_i$
- If f has at most k literals per clause then it is in k-CNF

CNF

Some of the most important computational problems concern logical formulas

- A logical formula f is said to be in conjunctive normal form (CNF) if

$$f = C_1 \wedge \ldots \wedge C_m$$

  where each C_i is a clause, which is a disjunction ($\vee$, "OR") of literals:

$$C_i = (l_{i1} \vee \ldots \vee l_{ik})$$

  and a literal is either a variable or its negation, e.g. x_i or $\neg x_i$
- If f has at most k literals per clause then it is in k-CNF
- For example:

$$(x_1 \vee x_2 \vee \neg x_5) \wedge (\neg x_2 \vee \neg x_4 \vee \neg x_5) \wedge (x_2 \vee x_3 \vee x_4)$$

  is in 3-CNF

Satisfiability

- The logical formula f is satisfiable if there exists an assignment of TRUE and FALSE to the variables of f which makes f TRUE .
- f is TRUE iff all clauses are TRUE
- a clause is TRUE iff at least one literal is TRUE
- For example:

$$(x_1 \vee x_2 \vee \neg x_5) \wedge (\neg x_2 \vee \neg x_4 \vee \neg x_5) \wedge (x_2 \vee x_3 \vee x_4)$$

is satisfied by this assignment (amongst others):

$$x_1 = \text{TRUE}\ , x_2 = \text{FALSE}\ , x_3 = \text{TRUE}\ , x_4 = \text{TRUE}\ , x_5 = \text{FALSE}\ .$$

The Satisfiability Problem

SATISFIABILITY
Instance: CNF formula f
Question: Is f satisfiable?

k-SATISFIABILITY
Instance: k-CNF formula f
Question: Is f satisfiable?

2-Satisfiability

Proposition

2-SATISFIABILITY *is in* **P**

2-Satisfiability

Proposition

2-SATISFIABILITY *is in* **P**

Proof:

0. Declare all clauses unsatisfied and literals unassigned.

2-Satisfiability

Proposition

2-SATISFIABILITY *is in* **P**

Proof:

0. Declare all clauses unsatisfied and literals unassigned.
1. Select an arbitrary unassigned variable x and assign x the value TRUE and $\neg x$ the value FALSE

2-Satisfiability

Proposition

2-SATISFIABILITY *is in* **P**

Proof:

0. Declare all clauses unsatisfied and literals unassigned.
1. Select an arbitrary unassigned variable x and assign x the value TRUE and $\neg x$ the value FALSE
2. Select an unsatisfied clause $l_i \vee l_j$,
   1. If both literals are unassigned, ignore the clause.
   2. If at least one literal is assigned TRUE , declare the clause satisfied.
   3. if one literal is FALSE and the other is unassigned, set the other to TRUE and its negation to FALSE , and declare the clause satisfied.
   4. If both are FALSE , restart the algorithm setting x FALSE and $\neg x$ TRUE . If a conflict occurs again, declare unsatisfiable.

Proof continued

3. Repeat step 2, until either
   1. all clauses are satisfied, return satisfiable;

Proof continued

3. Repeat step 2, until either
   1. all clauses are satisfied, return satisfiable;
   2. or all clauses remaining not satisfied (yet) have all their variables unassigned. In this case return to step 1;

Exercise: Complete the proof by showing that this algorithm is correct and has polynomial running time.

Polynomial-time reducibility

Another way to show that a problem is in **P** is to use a reduction.

Informally, a problem P is reducible to a problem Q if we can somehow use methods that solve Q in order to solve P.

Definition

Language $\mathcal{L}_1$ is polynomially reducible to $\mathcal{L}_2$, denoted $\mathcal{L}_1 \leq \mathcal{L}_2$, if a polynomially-bounded and polynomial-time computable function f exists such that: $x \in \mathcal{L}_1 \Leftrightarrow f(x) \in \mathcal{L}_2$

Polynomial-time reducibility

Another way to show that a problem is in **P** is to use a reduction.

Informally, a problem P is reducible to a problem Q if we can somehow use methods that solve Q in order to solve P.

Definition

Language $\mathcal{L}_1$ is polynomially reducible to $\mathcal{L}_2$, denoted $\mathcal{L}_1 \leq \mathcal{L}_2$, if a polynomially-bounded and polynomial-time computable function f exists such that: $x \in \mathcal{L}_1 \Leftrightarrow f(x) \in \mathcal{L}_2$

Recall: the (polynomial) reduction we saw from VERTEX COVER to SET COVER.

Polynomial-time reducibility

Another way to show that a problem is in **P** is to use a reduction.

Informally, a problem P is reducible to a problem Q if we can somehow use methods that solve Q in order to solve P.

Definition

Language $\mathcal{L}_1$ is polynomially reducible to $\mathcal{L}_2$, denoted $\mathcal{L}_1 \leq \mathcal{L}_2$, if a polynomially-bounded and polynomial-time computable function f exists such that: $x \in \mathcal{L}_1 \Leftrightarrow f(x) \in \mathcal{L}_2$

Recall: the (polynomial) reduction we saw from VERTEX COVER to SET COVER.

This fundamental tool of complexity theory was introduced by Richard Karp in 1972.
(one of the inventors of the Edmonds-Karp algorithm for max flow).

Key property of polynomial-time reducibility

Lemma

$$\mathcal{L}_1 \leq \mathcal{L}_2 \textit{ and } \mathcal{L}_2 \in \mathbf{P} \Longrightarrow \mathcal{L}_1 \in \mathbf{P}$$

Main idea: The composition of polynomials is a polynomial.

Key property of polynomial-time reducibility

Lemma

$$\mathcal{L}_1 \leq \mathcal{L}_2 \text{ and } \mathcal{L}_2 \in \mathbf{P} \Longrightarrow \mathcal{L}_1 \in \mathbf{P}$$

Main idea: The composition of polynomials is a polynomial.

Proof of the Lemma:

- Let A_2 be a polynomial-time algorithm that decides L_2
- Let f be a polynomial-time reduction algorithm from L_1 to L_2
- We construct a polynomial-time algorithm A_1 that decides L_1:

1. given input x of L_1, compute input $f(x)$ of L_2 in polynomial time (we know that $x \in L_1 \iff f(x) \in L_2$)
2. use algorithm A_2 to decide whether $f(x) \in L_2$
3. if $f(x) \in L_2$ then output YES; otherwise output NO

k-colorability

- Let $G = (V, E)$ be a graph, with vertices V and edges E.
- A function $f : V \to \{1, \ldots, n\}$ is a (proper) coloring if adjacent vertices are assigned different values (= colors).

k-COLORABILITY
Instance: A graph G
Question: Is there a coloring of G using at most k colors?

(When $k = 2$ this is equivalent to BIPARTITE)

2-COLORABILITY $\leq$ 2-SATISFIABILITY

We can reduce 2-COLORABILITY to 2-SATISFIABILITY:

- For each vertex v_i of the graph we create a variable x_i
- For each edge (v_i, v_j) we add two clauses $(x_i \vee x_j)$ and $(\neg x_i \vee \neg x_j)$.

2-COLORABILITY $\leq$ 2-SATISFIABILITY

We can reduce 2-COLORABILITY to 2-SATISFIABILITY:

- For each vertex v_i of the graph we create a variable x_i
- For each edge (v_i, v_j) we add two clauses $(x_i \vee x_j)$ and $(\neg x_i \vee \neg x_j)$.

This translation of a 2-COLORABILITY problem to a 2-SATISFIABILITY problem is computable in polynomial time. Now we check it satisfies the reducibility condition:

$\Rightarrow$ If the graph is 2-colorable, use 2-coloring to assign truth values tovariables (one color is TRUE , the other FALSE).

2-COLORABILITY $\leq$ 2-SATISFIABILITY

We can reduce 2-COLORABILITY to 2-SATISFIABILITY:

- For each vertex v_i of the graph we create a variable x_i
- For each edge (v_i, v_j) we add two clauses $(x_i \vee x_j)$ and $(\neg x_i \vee \neg x_j)$.

This translation of a 2-COLORABILITY problem to a 2-SATISFIABILITY problem is computable in polynomial time. Now we check it satisfies the reducibility condition:

$\Rightarrow$ If the graph is 2-colorable, use 2-coloring to assign truth values tovariables (one color is TRUE , the other FALSE).

$\Leftarrow$ If the formula is satisfiable, define 2-coloring by setting TRUE variables to color 1 and FALSE to color 2. If two adjacent vertices get the same color then one of the associated clauses is not satisfied (contradiction). Thus we have a proper 2-coloring.

An aside

- Some problems have been shown to be in **P** by non-constructive arguments (i.e., by proving a polynomial time algorithm must exist)
- e.g. The problem KNOTLESSNESS – can a given graph be embedded in 3-dimensional space without knots.
- In these cases we may not even know how to find an exponential algorithm!
- (See Bovet and Crescenzi for more discussion of this).